2

IF-ELSE AND FLOW CONTROL

You can make a program execute some instructions and skip others based on whether conditions evaluate to a Boolean `True` or `False` value. These activities test your ability to work with the `if`, `elif`, and `else` flow control statements.

LEARNING OBJECTIVES

- Understand Boolean values and their role in conditions.
- Master the comparison and Boolean operators and how to construct simple and more complicated conditions with them.
- Identify blocks of code based on their indentation level.
- Learn how to selectively execute code in different parts of your program with `if`, `elif`, and `else` statements.

Practice Questions

These questions test your ability to reason about Boolean values and operators used in conditions. If you get stuck trying to figure out what a question's expression evaluates to, try entering it into the interactive shell.

Boolean Values

The Boolean data type has only two values: `True` and `False`. They can be stored in variables and used in expressions, just like values of other data types. For each of the following, answer "yes" if it is a Python Boolean value and "no" if it is not.

1. `False`
2. `'True'`
3. `false`
4. `True`
5. `'false'`

6. `true`

Comparison Operators

Comparison operators, also called *relational operators*, compare two values and evaluate down to a Boolean `True` or `False`. For each of the following, answer "yes" if it is a Python comparison operator and "no" if it is not.

7. `=`
8. `<`
9. `=>`
10. `=!`
11. `!=`
12. `==`
13. `>`
14. `<=`

To test your understanding of Python's comparison operators, answer the following questions.

15. What is the difference between the `<` and `<=` operators?
16. What is the difference between the `=` and `==` operators?
17. Why does `42 == 42.0` evaluate to `True`?
18. Why does `42 == '42'` evaluate to `False`?
19. What happens if you enter `42 < 'hello'` into the interactive shell?

Boolean Operators

The three Boolean operators (`and`, `or`, and `not`) are used to compare Boolean values. Like comparison operators, they evaluate expressions down to a Boolean `True` or `False` value.

Draw the truth tables for the Boolean operators in questions 20 through 22.

20. `and`
21. `or`
22. `not`

What do the following expressions evaluate to?

23. `2 + 2 > 4 or True`
24. `True and 2 + 2 >= 4`
25. `True and (True or False)`
26. `(False or True) and True`
27. `True and not False`

28. `not (False or True)`

29. `not False or True`

30. `True and True and True and True and False`

31. `False or False or False or True or False`

You can use Boolean operators in an expression along with the comparison operators to evaluate the value of a variable. Answer the following questions.

32. Say the variable `is_raining` is set to either `True` or `False`. Describe what the assignment statement `is_raining = not is_raining` does.

33. If the variable `name` has the value `'Alice'`, which expression is correct: the expression `name == 'Alice' or name == 'Bob'` or the expression `name == 'Alice' or 'Bob'`?

Components of Flow Control

Flow control statements often start with a part called the *condition* and are always followed by a block of code called the *clause*. A flow control statement decides what to do based on whether its condition is `True` or `False`, and almost every flow control statement uses a condition. Python code can be grouped together in blocks, which you can identify by their level of indentation.

34. When does a new block begin?

35. Can a block be inside another block?

36. A new block is expected after statements that end with what character?

37. When does a block end?

38. What is the program execution?

Questions 39 through 41 relate to the following program (which is labeled with line numbers):

```
1. name = 'Alitza'
2. if name == 'Dolly':
3.     print('Hello, Dolly!')
4. print('Done')
```

39. How many blocks are in this program?

40. On what line does the first block begin?

41. On what line does the first block end?

Flow Control Statements

The most common type of flow control statement is the `if` statement. An `if` statement's clause (that is, the block following the `if` statement) will execute if the statement's condition is `True` and be skipped if the condition is `False`. The `else` and `elif` statements can follow `if` statements with other instructions.

Answer "yes" if the following are valid `if` statements, given a variable named `eggs` that has the value 12. Answer "no" if they are not valid statements.

42. `if eggs = 12:`

43. `if eggs > 12`

44. `if:`

45. `if eggs == 12:`

46. `if eggs != 'hello':`

47. `if eggs < 12:`

Answer "yes" if the following are valid `else` statements, given that they follow a statement `if eggs == 12:` and its `if` block. Answer "no" if they are not valid statements.

48. `else:`

49. `else if eggs != 12:`

50. `else`

51. `else if not:`

52. `else not:`

Answer "yes" if the following are valid `elif` statements, given that they follow a statement `if eggs == 12:` and its `if` block. Answer "no" if they are not valid statements.

53. `elif:`

54. `elif eggs != 12:`

55. `else if eggs != 12:`

56. `elif eggs == 12:`

Examine the following flawed program:

```
password = 'swordfish'
if password == 'rosebud':
    print('Access granted.')
else:
    print('Access denied.')
elif password == 'swordfish':
    print('That is the old password.')
```

57. Why does this program cause an error?

58. How many `elif` statements (each with an `elif` block of code following it) can follow an `if` statement and an `if` block of code?

Practice Projects

Work with the following short programs and write one of your own to demonstrate the concepts you've learned in this chapter.

Fixing the Safe Temperature Program

The following program reports whether a given temperature is safe. It asks the user to enter a temperature in two parts. First, they should enter `C` or `F` to indicate the Celsius or Fahrenheit scale; second, they should enter the number of degrees. If the temperature is between `16` and `38` degrees Celsius (inclusive of `16` and `38`) or between `60.8` and `100.4` degrees Fahrenheit (inclusive of `60.8` and `100.4`), the program prints `Safe`. Outside of these temperature ranges, the program prints `Dangerous`.

This program has bugs, however. Rewrite the code to fix the errors. You may assume the user always enters valid inputs and not, say, `X` for the scale or `hello` for the number of degrees.

```
print('Enter C or F to indicate Celsius or Fahrenheit:')
scale = input()
print('Enter the number of degrees:')
degrees = int(input())
if scale == 'C':
    if degrees >= 16 or degrees <= 38:
        print('Dangerous')
    else:
        print('Dangerous')
elif scale == 'F':
    if degrees > 60.8 and degrees >= 100.4:
        print('Safe')
    else:
        print('Dangerous')
```

Test this program by entering a temperature in both the safe and dangerous ranges and in both the Celsius and Fahrenheit scales.

Save this program in a file named *safeTemp.py*.

Single-Expression Safe Temperature

It's possible to write the safe temperature logic of the previous program in a single condition. Fill in the blank in the following program with this condition to make it work in the same way as the previous program:

```
print('Enter C or F to indicate Celsius or Fahrenheit:')
scale = input()
print('Enter the number of degrees:')
degrees = int(input())
if ____:
    print('Safe')
else:
    print('Dangerous')
```

This condition will be rather long. As a hint, you'll need to have separate parts for Celsius and Fahrenheit, combined by an `or` operator. It should look something like this: `(scale == 'C' and ____) or (scale == 'F' and ____)`.

Test this program by entering a temperature in both the safe and dangerous ranges and in both the Celsius and Fahrenheit scales.

Save this program in a file named *safeTempExpr.py*.

Fizz Buzz

Fizz Buzz is a common programming challenge that goes like this. Write a program that accepts an integer from the user. If the integer is divisible by `3`, the program should print `Fizz`. If the integer is divisible by `5`, the program should print `Buzz`. If the integer is divisible by `3` and `5`, the program should print `Fizz Buzz`. Otherwise, the program should print the number the user entered. The output of this program should look something like this:

```
Enter an integer:
18
Fizz
```

Or this:

```
Enter an integer:
25
Buzz
```

Or this:

```
Enter an integer:
15
Fizz Buzz
```

Or this:

```
Enter an integer:
37
37
```

Here are some hints to help you write this program:

- Use the modulo operator to determine whether a number is divisible. If the condition `number % 3 == 0` is `True`, then `number` is divisible by `3`.
- Be sure to check whether the number is divisible by both `3` and `5` before checking whether the number is divisible by either `3` or `5`. Otherwise, the number `15` won't cause the program to print `Fizz Buzz`.

Save this program in a file named *fizzBuzzNumber.py*.